

BEAM
SUMMIT

Using Python Memory Profilers with Apache Beam

Valentyn Tymofieiev
Google
tvalentyn@apache.org



Outline



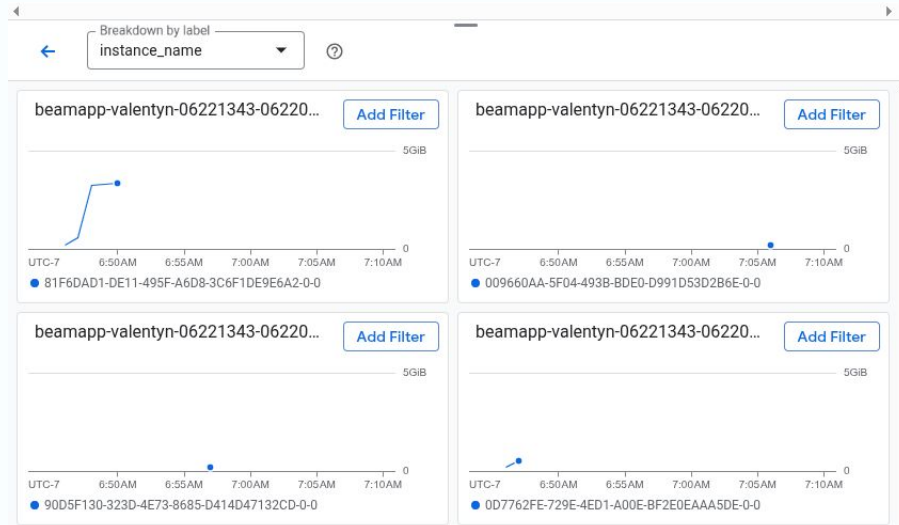
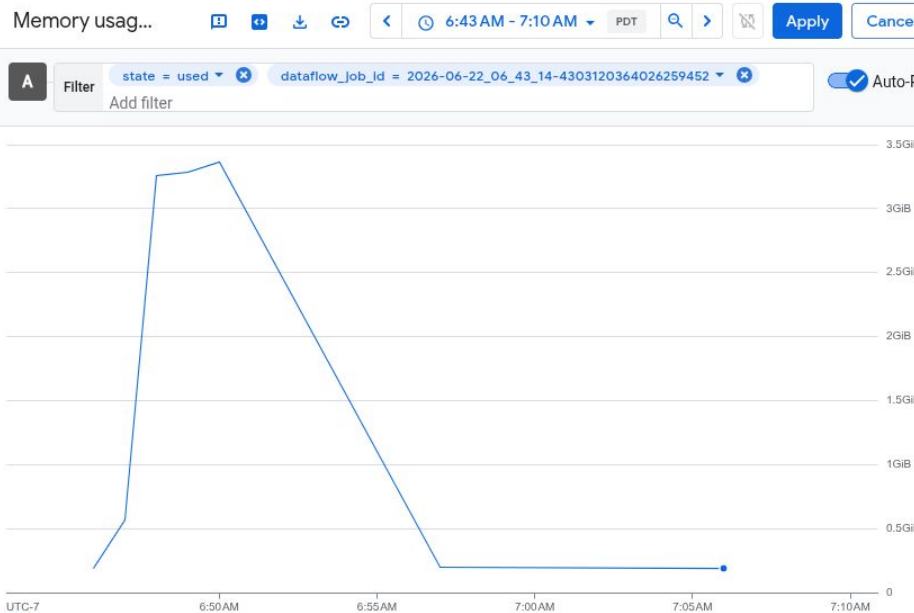
- What does it take to run a Python pipeline with a profiler
- Using Memray to profile Beam pipelines
- How to tune profiling options



Why?



Runners might capture per-worker memory usage but offer no insights into user code profiling.





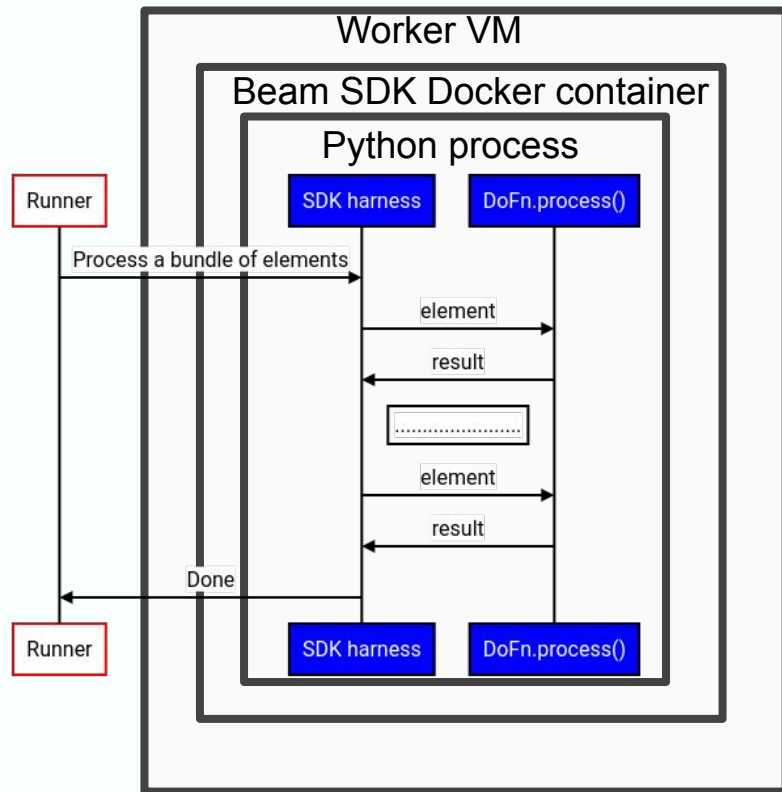
How?



Collect -> Store -> Post-process -> Insights



Beam Python worker runtime





Worker logs at pipeline startup



- ⚙ Initializing python harness: `/opt/apache/beam/boot --logging_endpoint=localhost:12370 --control_endpoint=localhost:12371`
- ⚙ Installing user dependencies ...
- ⚙ Executing Python (WORKER_ID=sdk-0-0): `python -m apache_beam.runners.worker.sdk_worker_main`
- ⓘ Creating insecure control channel for localhost:12371.
- ⓘ Control channel established.



Instrumenting Python SDK harness w/ profiler



Default launch command:

```
⚙ Executing Python (WORKER_ID=sdk-0-0): python -m apache_beam.runners.worker.sdk_worker_main
```

New Pipeline Options:

- `--profiler_agent=` [memray, tcmalloc, <custom_agent>]
- `--profiler_extra_arg=` [args to pass to the profiler]
- `--profiler_extra_env_var=` [additional env variables to set]

Transformed launch command:

```
$ ENV_KEY=VAL profiler [profiler_args] python -m <sdk_worker_main>
```




Collect -> **Store** -> Post-process -> Insights



Standalone

- Stores data **locally** on the host machine.
- Memray, Gperftools Profiler
- Requires extra disk space
- Beam must preserve profiles



Continuous

- Transmits metrics to an **external backend**.
- Parca, Pyroscope, Datadog, Google Cloud Profiler



grafana / pyroscope

<> Code Issues 357 Pull requests 29 Discussions Actions Projects

Memory profiling on python #316

Open



arunthamma opened on Aug 4, 2021

Hi,

I have started using pyroscope for profiling cloud applications developed only profiling CPU. Can I get some information on integrating memory profiling?

Regards.



parca-dev / parca-agent

<> Code Issues 169 Pull requests 6 Discussions Actions Security

Python Memory Profiling #2643

Open



yaroslavkasatikov opened on Mar 25, 2024

Hi team,
I have tried to lookup many docs, but haven't found the way how to do it. Is it possible (ideally I want to have smth like [bloomberg/memray](#) but with support for Python)? Could you please help me with this case?

Thanks!



Profile types for Python:

- CPU time **...what about RAM?**
- Wall time (main thread)

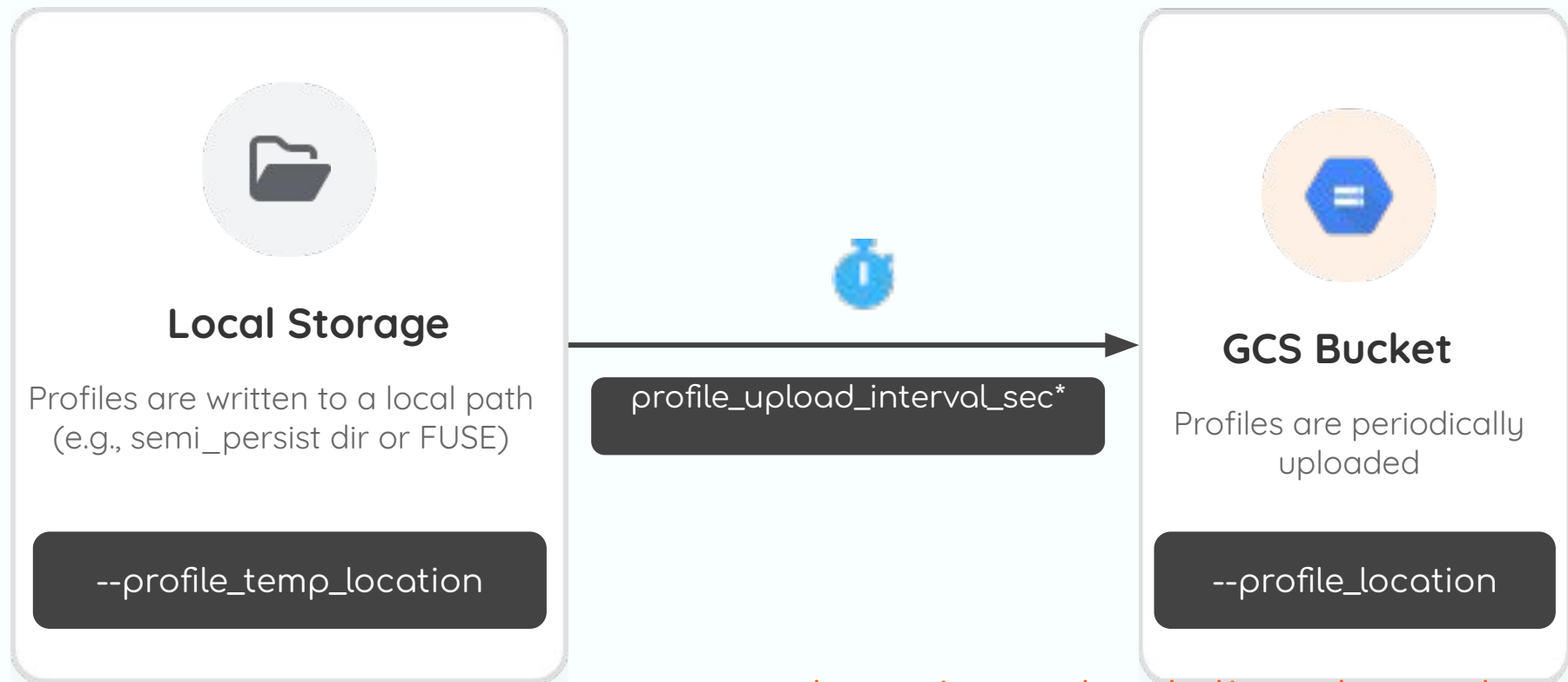


Memray specifics



- Profiles individual processes
 - Enable with: `--profiler_agent=memray`
- Saves output locally
- Requires postprocessing
 - `memray flamegraph memray-allocations.bin -o human_readable_graph.html`
- Notable modes
 - Native code profiling: `--profiler_extra_arg="--native"`
 - Aggregate mode:
 - Smaller profiles but requires process to gracefully shutdown
 - `--profiler_extra_arg="--aggregate"`
`--profiler_stop_after_sec=600`
- Learn more: <https://bloomberg.github.io/memray/>

🔍 GCS storage for standalone profilers



* requires gcloud cli on the workers



Collect -> Store -> **Post-process** -> Insights



On-the-worker post-processing

- Peak Heap Allocation Flamegraph
- Leaks Flamegraph
- `--profile_postprocess_interval_sec`

Offline post-processing

- Possible but may require replicating the same runtime environment.
 - Could be done using a separate Beam pipeline using the same `--sdk_container_image`



Saved artifacts after postprocessing + upload



Objects

Configuration

Permissions

Protection

Lifecycle

Observability

Inventory Reports

Operations

I>

Buckets > valentyn-test > profiles > 2026-06-16_17_20_55-2731622616648424593 > beamapp-valentyn-06170020-061

job id

worker name

Create folder

Upload ▾

Transfer data ▾

Create batch operations

New

Other services ▾

Filter by name prefix only ▾

Filter

Filter objects and folders

☐	Name	Size	Type	Created ?
☐	 memray-sdk-0-0-20260617002402.bin	8 MB	application/octet-stream	Jun 16, 2026, 5:34:02 PM
☐	 memray-sdk-0-0-20260617002402.html	8.3 MB	text/html	Jun 16, 2026, 5:34:02 PM
☐	 memray-sdk-0-0-20260617002402_leaks.html	8.4 MB	text/html	Jun 16, 2026, 5:34:02 PM
☐	 memray-sdk-0-0_sibling_1-20260617002402.bin	8.8 MB	application/octet-stream	Jun 16, 2026, 5:34:02 PM
☐	 memray-sdk-0-0_sibling_1-20260617002402.html	8.4 MB	text/html	Jun 16, 2026, 5:34:01 PM
☐	 memray-sdk-0-0_sibling_1-20260617002402_leaks.html	8.4 MB	text/html	Jun 16, 2026, 5:34:01 PM

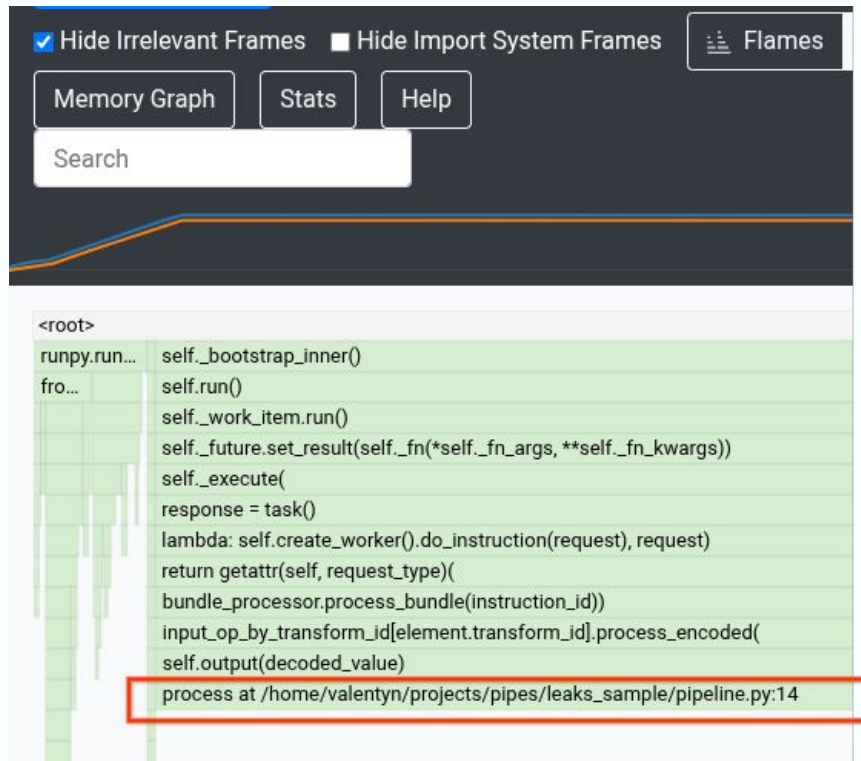
<https://console.cloud.google.com/storage/browser/bucket/profiles>



Collect -> Store -> Post-process -> **Insights**



Sample Memray flamegraph



gs://bucket/profiles/JOB_ID/HOSTNAME/memray-sdk-0-0_sibling_#-timestamp.html



Sample Memray flamegraph, --native mode



memray flamegraph report

Python Allocator: pymlibc

Hide Irrelevant Frames

Hide Import System Frames

```

<root>
runpy.run_module(argv.script, ...
builtins.__import__ at Python/...
builtins.__import__ at Python/...
PyImport_ImportModuleLevel...
import_find_and_load at Pyth...

self.run()
self._work_item.run()
self._future.set_result(self._fn(*self._fn_args, **self._fn_kwargs))
self._execute()
response = task()
lambda: self.create_worker() do_instruction(request, request)
return getattr(self, request_type)(
bundle_processor.process_bundle(instruction_id))
input_op_by_transform_id(element.transform_id).process_encoded(
self.output(decoded_value))
...pyx_pw_11apache_beam_runners_6worker_10operations_90operation_30output at apache_beam/runners/worker/operations.c:20328
...pyx_pf_11apache_beam_runners_6worker_10operations_90operation_30output at apache_beam/runners/worker/operations.c:20359
...pyx_f_11apache_beam_runners_6worker_10operations_90operation_output at apache_beam/runners/worker/operations.c:20201
...pyx_pw_11apache_beam_runners_6worker_10operations_275SingletonElementConsumerSet_3receive at apache_beam/runners/worker/operations.c:12390
...pyx_pf_11apache_beam_runners_6worker_10operations_275SingletonElementConsumerSet_3receive at apache_beam/runners/worker/operations.c:12418
...pyx_f_11apache_beam_runners_6worker_10operations_275SingletonElementConsumerSet_receive at apache_beam/runners/worker/operations.c:12275
...pyx_f_11apache_beam_runners_6worker_10operations_110DoOperation_process at apache_beam/runners/worker/operations.c:31404
...pyx_f_11apache_beam_runners_6common_10DoFnRunner_process at apache_beam/runners/common.c:37308
...pyx_f_11apache_beam_runners_6common_13SimpleInvoker_invoke_process at apache_beam/runners/common.c:21342
...pyx_f_11apache_beam_runners_6common_14_OutputHandler_handle_process_outputs at apache_beam/runners/common.c:43186
...pyx_f_11apache_beam_runners_6common_14_OutputHandler_write_value_to_tag at apache_beam/runners/common.c:45110
...pyx_f_11apache_beam_runners_6worker_10operations_275SingletonElementConsumerSet_receive at apache_beam/runners/worker/operations.c:12275
...pyx_f_11apache_beam_runners_6worker_10operations_110DoOperation_process at apache_beam/runners/worker/operations.c:31404
...pyx_f_11apache_beam_runners_6common_10DoFnRunner_process at apache_beam/runners/common.c:37308
...pyx_f_11apache_beam_runners_6common_13SimpleInvoker_invoke_process at apache_beam/runners/common.c:21342
...pyx_f_11apache_beam_runners_6common_14_OutputHandler_handle_process_outputs at apache_beam/runners/common.c:43186
...pyx_f_11apache_beam_runners_6common_14_OutputHandler_write_value_to_tag at apache_beam/runners/common.c:45110
...pyx_f_11apache_beam_runners_6worker_10operations_275SingletonElementConsumerSet_receive at apache_beam/runners/worker/operations.c:12275
...pyx_f_11apache_beam_runners_6worker_10operations_110DoOperation_process at apache_beam/runners/worker/operations.c:31404
...pyx_f_11apache_beam_runners_6common_10DoFnRunner_process at apache_beam/runners/common.c:37308
...pyx_f_11apache_beam_runners_6common_13SimpleInvoker_invoke_process at apache_beam/runners/common.c:21342
...pyx_f_11apache_beam_runners_6common_14_OutputHandler_handle_process_outputs at apache_beam/runners/common.c:43097
gen_itemnext at Objects/genobject.c:603
gen_send_ex2 at Objects/genobject.c:230
process at /home/valentyn/projects/pipes/leaks_sample/pipeline.py:14
cfuction_call at Objects/methodobject.c:548
leak_memory at /tmp/pip-req-build-e_dxcpu/leaky_module.cpp:10
operator new(unsigned long) at <unknown>:0

```

```

__pyx_f_11apache_beam_7runners_6common_13SimpleInvoker_invoke_proce
__pyx_f_11apache_beam_7runners_6common_14_OutputHandler_handle_pro
gen_iternext at Objects/genobject.c:603
gen_send_ex2 at Objects/genobject.c:230
process at /home/valentyn/projects/pipes/leaks_sample/pipeline.py:14
cfuction_call at Objects/methodobject.c:548
leak_memory at /tmp/pip-req-build-e_dxcpu/leaky_module.cpp:10
operator new(unsigned long) at <unknown>:0

```



- Basic usage

```
python your_pipeline.py --profiler_agent=memray
```

- Customize profiler and Beam instrumentation via options

- `--profiler_agent=your_custom_profier`
- `--profiler_extra_arg=...`
- Examples <https://github.com/apache/beam/pull/38853>

- Instrumentation available in Beam 2.75.0



Memray reqs for custom container users



- Use Beam 2.75.0+ SDK and container entrypoint
- Install **memray** and **gcloud** cli

```
FROM python:3.10-slim
```

```
RUN pip install --no-cache-dir apache-beam[gcp]==2.75.0 memray==1.19.3
```

```
# Install Google Cloud SDK.
```

```
RUN \  
    mkdir -p /usr/local/gcloud && \  
    cd /usr/local/gcloud && \  
    curl -s -O https://dl.google.com/dl/cloudsdk/release/google-cloud-sdk.tar.gz && \  
    tar -xf google-cloud-sdk.tar.gz && \  
    /usr/local/gcloud/google-cloud-sdk/install.sh && \  
    rm -rf /usr/local/gcloud/google-cloud-sdk/.install/.backup && \  
    rm google-cloud-sdk.tar.gz
```

```
COPY --from=apache/beam_python3.10_sdk:2.75.0 /opt/apache/beam /opt/apache/beam
```

```
ENTRYPOINT ["/opt/apache/beam/boot"]
```



Next steps



- Try it out!
- Provide feedback: <https://github.com/apache/beam/issues/20298>
- Learn more: <https://s.apache.org/beam-python-memory-profiling>

Valentyn Tymofieiev
tvalentyn@apache.org

Slides:

QUESTIONS?



<https://s.apache.org/profiling-beam-pipelines>